

Algorithms and Data Structures

26.06.2024

Exercise 1 (1.5 points)

Given the following array of integer values, perform the first step of quicksort to sort the array in ascending order; then, from the initial array, generate the right and the left partitions.

13 5 8 11 12 0 1 7 2 3

Report 3 integer values: The pivot selected on the original array, the pivot you would select on the left partition generated from the original array, and the pivot you would select on the right partition generated (again) from the original array. No other symbols must be included in the response. The following is an example of the response format:
13 1 10

Solution

Quick sort

Solution format 1:

```
Initial array: 13 5 8 11 12 0 1 7 2 3
Re l= 0;r= 2: 2 1 0
Re l= 0;r=-1:
Re l= 0;r=-1: 1 2
Re l= 1;r= 1: 1
Re l= 1;r= 1:
Re l= 0;r= 2: 12 8 5 7 13 11
Re l= 4;r= 6: 7 8 5
Re l= 4;r= 3:
Re l= 4;r= 3: 8 7
Re l= 5;r= 4:
Re l= 5;r= 4: 8
Re l= 4;r= 6: 13 12
Re l= 8;r= 7:
Re l= 8;r= 7: 13
```

Solution format 2:

```
Input array: [0]13 [1]5 [2]8 [3]11 [4]12 [5]0 [6]1 [7]7 [8]2 [9]3
Pivot: [9]3
Swaps: [0<->8]13<->2 - [1<->6]5<->1 - [2<->5]8<->0 - [3<->9]11<->3
Output array: [0]2 [1]1 [2]0 - [3]3 - [4]12 [5]8 [6]5 [7]7 [8]13 [9]11
Input array: [0]2 [1]1 [2]0 - [4]12 [5]8 [6]5 [7]7 [8]13 [9]11
Pivot: [2]0 - [9]11
Swaps: [0<->2]2<->0 - [4<->7]12<->7 - [7<->9]12<->11
Output array: - [0]0 - [1]1 [2]2 - [4]7 [5]8 [6]5 - [7]11 - [8]13 [9]12
Input array: [1]1 [2]2 - [4]7 [5]8 [6]5 - [8]13 [9]12
Pivot: [2]2 - [6]5 - [9]12
Swaps: [2<->2]2<->2 - [4<->6]7<->5 - [8<->9]13<->12
Output array: [1]1 - [2]2 - - - [4]5 - [5]8 [6]7 - - [8]12 - [9]13
Input array: [5]8 [6]7
Pivot: [6]7
Swaps: [5<->6]8<->7
Output array: - [5]7 - [6]8
```

Response

3 0 11

Exercise 2 (1.5 points)

A BST contains integer values included in the range 1-1000. Suppose we are looking for the value 711 is such a BST. Consider the following sequences of values generated during a search.

77 945 356 854 610 665 684 843 708 753 733 727 724 710 721 716 711
214 551 742 602 668 689 710 740 729 717 716 709 711
666 938 919 899 699 795 788 753 704 801 715 706 711
70 839 72 583 779 681 705 732 726 712 708 711

Indicate which one of the following sequences (numbered 1, 2, 3, and 4) is correct. Notice that wrong responses imply a mark penalty.

Solution

BST verify sequence

Searching: 711

```
Sequence 1: 77 945 356 854 610 665 684 843 708 753 733 727 724 710 721 716 711
            77 945 356 854 610 665 684 843 708 753 733 727 724 710 721 716 711 Sequence OK.
Sequence 2: 214 551 742 602 668 689 710 740 729 717 716 709 711
            214 551 742 602 668 689 710 Error: 709<710
Sequence 3: 666 938 919 899 699 795 788 753 704 801 715 706 711
```

```

666 938 919 899 699 795 Error: 801>795
Sequence 4: 70 839 72 583 779 681 705 732 726 712 708 711
              70 839 72 583 779 681 705 732 726 712 708 711 Sequence OK.

```

Response

1 4

Exercise 3 (1.5 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=23$. Insertions occur character by character using open addressing with double hashing. Use functions $h_1(k) = k \% M$ and $h_2(k) = 1 + (k \% 97)$. Each character is identified by its index in the English alphabet (i.e., A=1, ..., Z=26). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

R A N R A N

Indicate in which elements the last three letters of the sequence are placed, i.e., R, A, and N, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Solution

```

### hash-table
Character keys
Hash table size = 23
Number of keys = 6
Double hashing.
Key: R A N R A N
Key=R k=018 Final=18
Key=A k=001 Final=01
Key=N k=014 Final=14
Key=R k=018 Coll.=18 Coll.=14 Final=10
Key=A k=001 Coll.= 1 Final=03
Key=N k=014 Coll.=14 Final=06
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22
- A - A - - N - - - R - - - N - - - R - - - -

```

Response

10 3 6

Exercise 4 (4.0 points)

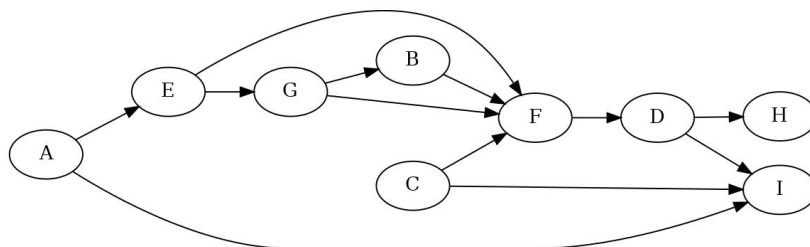
Define what the term “greedy algorithm” means in computer science. Describe (in English terms) the activity selection problem and how it can be solved (step-by-step). Report an example with the related resolution process and the final solution. Describe and motivate all main steps.

Solution

Please see the teacher's overhead for a complete analysis of the topic.

Exercise 5 (1.5 points)

Visit the following graph depth-first, starting at node A. Label nodes with discovery and end-processing times. Start with the discovery time set to 1 on A. When necessary, consider nodes and edges in alphabetic order.



Display the end-processing time of all vertices. Please indicate the end-processing time of all vertices sorted in alphabetic order (i.e., display the end-processing time for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 15 13 2 16 8 etc.

Solution

```

Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I
Adjacency Matrix:
0 0 0 0 1 0 0 0 1
0 0 0 0 0 1 0 0 0
0 0 0 0 0 1 0 0 1
0 0 0 0 0 0 0 1 1
0 0 0 0 0 1 1 0 0
0 0 0 1 0 0 0 0 0
0 1 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
### Graph visit
BFS:
Node=[ 0] A Discovery_time= 0 Predecessor=[-1]
Node=[ 1] B Discovery_time= 3 Predecessor=[ 6]
Node=[ 2] C Discovery_time= - Predecessor=[-1]
Node=[ 3] D Discovery_time= 3 Predecessor=[ 5]
Node=[ 4] E Discovery_time= 1 Predecessor=[ 0]
Node=[ 5] F Discovery_time= 2 Predecessor=[ 4]
Node=[ 6] G Discovery_time= 2 Predecessor=[ 4]
Node=[ 7] H Discovery_time= 4 Predecessor=[ 3]
Node=[ 8] I Discovery_time= 1 Predecessor=[ 0]
DFS:
List of edges:
[ 0] A -> [ 4] E : (T) Tree
[ 4] E -> [ 5] F : (T) Tree
[ 5] F -> [ 3] D : (T) Tree
[ 3] D -> [ 7] H : (T) Tree
[ 3] D -> [ 8] I : (T) Tree
[ 4] E -> [ 6] G : (T) Tree
[ 6] G -> [ 1] B : (T) Tree
[ 1] B -> [ 5] F : (C) Cross
[ 6] G -> [ 5] F : (C) Cross
[ 0] A -> [ 8] I : (F) Forward
[ 2] C -> [ 5] F : (C) Cross
[ 2] C -> [ 8] I : (C) Cross
List of vertices:
[ 0] A: dist_time= 1 endp_time=16 pred=[-1] -
[ 1] B: dist_time=12 endp_time=13 pred=[ 6] G
[ 2] C: dist_time=17 endp_time=18 pred=[-1] -
[ 3] D: dist_time= 4 endp_time= 9 pred=[ 5] F
[ 4] E: dist_time= 2 endp_time=15 pred=[ 0] A
[ 5] F: dist_time= 3 endp_time=10 pred=[ 4] E
[ 6] G: dist_time=11 endp_time=14 pred=[ 4] E
[ 7] H: dist_time= 5 endp_time= 6 pred=[ 3] D
[ 8] I: dist_time= 7 endp_time= 8 pred=[ 3] D
Reachable nodes from vertex [ 0] A:
[ 0] A - [ 1] B - [ 2] C - [ 3] D - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I -
UN-reachable nodes from vertex [ 0] A:
none

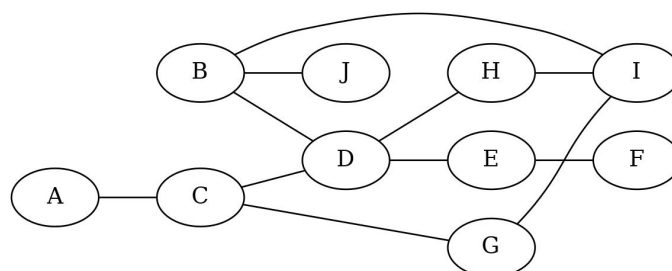
```

Response

16 13 18 9 15 10 14 6 8

Exercise 6 (1.5 points)

Given the following graph find all articulation points. If necessary, consider nodes and edges in alphabetical order.



Display all articulation points alphabetically. Please report only the list of vertices separated by a single space. No other symbols must be included in the response. The following is an example of the response format: A B C etc.

Solution

```

UN-directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J
Adjacency Matrix:
0 0 1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 1
0 0 0 1 0 0 0 0 0 1

```

```

1 0 0 1 0 0 1 0 0 0
0 1 1 0 1 0 0 1 0 0
0 0 0 1 0 1 0 0 0 0
0 0 0 0 1 0 0 0 0 0
0 0 1 0 0 0 0 0 1 0
0 0 0 1 0 0 0 0 1 0
0 1 0 0 0 0 1 1 0 0
0 1 0 0 0 0 0 0 0 0
### Graph Articulation Points and Bridges
Bridge: Edge [ 1] B - [ 9] J
Bridge: Edge [ 4] E - [ 5] F
Bridge: Edge [ 3] D - [ 4] E
Bridge: Edge [ 0] A - [ 2] C
Articulation point: Vertex [ 1] B
Articulation point: Vertex [ 2] C
Articulation point: Vertex [ 3] D
Articulation point: Vertex [ 4] E

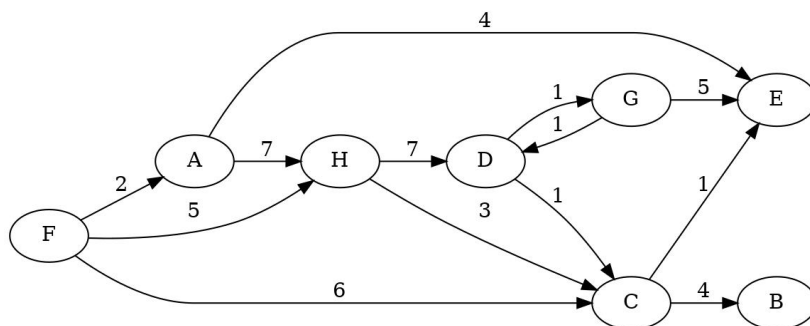
```

Response

B C D E

Exercise 7 (1.5 points)

Given the following directed and weighted graph, apply Dijkstra's algorithm to find all shortest paths connecting node F with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please indicate the shortest path to all vertices sorted in alphabetic order (i.e., display the shortest path for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8 etc.

Solution

```

Directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H
Adjacency Matrix:
0 0 0 0 4 0 0 7
0 0 0 0 0 0 0 0
0 4 0 0 1 0 0 0
0 0 1 0 0 0 1 0
0 0 0 0 0 0 0 0
2 0 6 0 0 0 0 5
0 0 0 1 5 0 0 0
0 0 3 7 0 0 0 0
### Graph SSSP (Dijkstra and Bellman-Ford)
Dijkstra
Shortest paths from vertex [ 5] F
Node [ 5] F: 0 (pred=-1)
Node [ 0] A: 2 (pred= 5)
Node [ 7] H: 5 (pred= 5)
Node [ 2] C: 6 (pred= 5)
Node [ 4] E: 6 (pred= 0)
Node [ 1] B: 10 (pred= 2)
Node [ 3] D: 12 (pred= 7)
Node [ 6] G: 13 (pred= 3)
Bellman Ford
Iteration 0: A:2 B:infy C:6 D:12 E:infy F:0 G:infy H:5
Iteration 1: A:2 B:10 C:6 D:12 E:6 F:0 G:13 H:5
Iteration 2: A:2 B:10 C:6 D:12 E:6 F:0 G:13 H:5
Shortest paths from vertex [ 5] F
Node [ 0] A: 2 (pred= 5)
Node [ 1] B: 10 (pred= 2)
Node [ 2] C: 6 (pred= 5)
Node [ 3] D: 12 (pred= 7)
Node [ 4] E: 6 (pred= 0)
Node [ 5] F: 0 (pred=-1)
Node [ 6] G: 13 (pred= 3)

```

Node [7] H: 5 (pred= 5)

Response

2 10 6 12 6 0 13 5

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```
int main(void) {
    char *s1[]={ "first", "second", "third", NULL}, *s2;
    int i, j, k, n;
    for (i=n=0; s1[i]!=NULL; i++)
        n += strlen (s1[i]);
    s2 = malloc ((n+1) * sizeof(char));
    k = 0;
    for (i=0; s1[i]!=NULL; i++) {
        for (j=strlen(s1[i])-1; j>=0; j--) {
            s2[k]=s1[i][j];
            k++;
        }
    }
    s2[k] = '\0';
    printf ("%s", s2);
    return 0;
}
```

Response

tsrifdnocesdriht

Exercise 9 (2.0 points)

The following function deletes the node with the given key from a BST rooted at the *root*.

```
node_t *delete_r (node_t *root, char *key) {
    node_t *p;
    char *val;
    if (root == NULL)
        return (root);
    // LINE 1
    root->l = delete_r (root->l, key);
    return (root);
}
if (strcmp (key, root->key) > 0) {
    root->r = delete_r (root->r, key);
    return (root);
}
p = root;
if (root->r == NULL) {
    root = root->l;
    free (p);
    return (root);
}
if (root->l == NULL) {
    root = root->r;
    free (p);
    return (root);
}
root->l = f (&val, root->l);
root->key = val;
return (x); // LINE 2
}
node_t *f (char *val, node_t *root) {
    node_t tmp;
    if (root->r == NULL) {
        *val = root->key;
        tmp = x; // LINE 3
        free (root);
        return (tmp);
    }
    root->r = f (val, root->r);
    return (root);
}
```

Indicate which one of the following statements is correct. Note that incorrect answers imply a penalty in the final score.

1. Even if the BST is not balanced, the function has a time complexity equal to $O(h)$ where h is the height of the tree.
2. Even if the BST is not balanced, the function has a time complexity equal to $O(\log(n))$ where n is the number of nodes stored in the tree.
3. Line 1 must include the test `if (strcmp (key, root->key) <= 0)`

4. In Line 2, `x` must be substituted by `val`.
5. The function `f` finds the maximum value in the left subtree.
6. The function `f` finds the minimum value in the right subtree.
7. In Line 3, `x` must be equal to `root->l`.
8. In Line 3, `x` must be equal to `root->r`.

Response

1. TRUE.
2. FALSE.
3. FALSE.
4. FALSE.
5. TRUE.
6. FALSE.
7. TRUE.
8. FALSE.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```
#define N 100
void f (char []);
int main (void){
    char s[N] = "1.This2.Is3.A4.String";
    f(s);
    printf ("%s", s);
    return 0;
}
void f (char *s) {
    if (*s=='\0')
        return;
    if (*s>='a' && *s<='z') {
        *s = *s + (int) ('A'-'a');
    } else {
        if (*s>='A' && *s<='Z') {
            *s = *s + (int) ('a'-'A');
        } else {
            if ((*s>='0' && *s<='9') || *s=='.')
                *s = '_';
        }
    }
    f (s+1);
    return;
}
```

Response

__tHIS__iS__a__sTRING

Exercise 11 (3.0 points)

Write the function

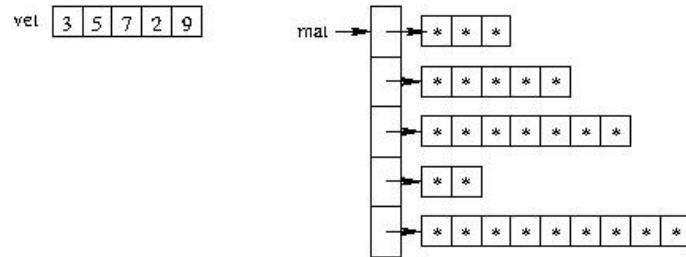
```
char **array_to_histogram (int *vet, int n);
```

which receives an array of `n` integers `vet`, and it dynamically allocates and returns the pointer to a 2D matrix. Such a matrix includes `n` rows and on each row `i` is the number of star symbols (*) equal to the value of the array element `vet[i]`.

The following picture represents a sample of the input array and the corresponding output matrix.

The pointer of such a matrix must be returned by the function `array_to_histogram`.

Write the entire program using standard C libraries but implement all required personal libraries. Notice that rows are simple arrays of characters not C strings (i.e., they do not have to be NULL terminated). Modularize the program



adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.

Exercise 12 (5.0 points)

A file includes an undefined number of rows. On each row the first field is a string (of a maximum of 100 characters), and the second an integer value that indicates the number of integer values following it on the same file line. For example, the following is a correct file with this format:

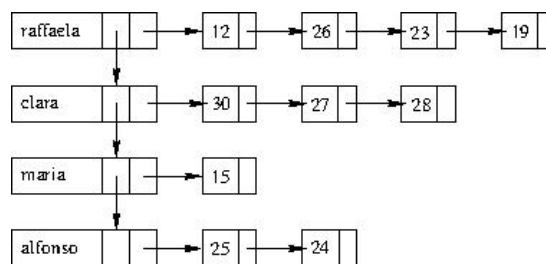
```
alfonso 2 24 25
maria 1 15
clara 3 28 27 30
raffaella 3 19 23 26
clara 2 27 28
raffaella 2 12 23
```

Write the function:

```
list1_t *file_to_list_of_lists (char *name);
```

which receives the file name as a parameter and returns the pointer to a list of lists storing the file content. The rules to store the file into a list of lists are as follows. The main list stores the strings in any order but without duplication. If a string appears more than once in the file (e.g., raffaella, as in the previous example) the corresponding numbers must be concatenated in the same secondary list. All secondary lists store the values associated with the corresponding strings. Those values may appear in any order but without duplication.

The following picture represents a sample of the input array and the corresponding output matrix.



Thus, for example, the secondary list related to raffaella must include the value {19, 23, 26, 12} in any order as 19 appear twice and must be inserted only once in the associated list.

Define the type `list1_t` and `list2_t` for the primary and secondary lists. The string within the `list1_t` type must be dynamically allocated.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.

Exercise 13 (9.0 points)

A file contains an integer n on the first row, and n strings on the following rows. Each string does contain small letters, capital letters, and decimal digits. An alphanumeric acronym of length n is composed by selecting for each element i of *acronym*[i] a character from the i -th string.

Write the recursive function:

```
void acronym (char *nameI, char *nameO);
```

that:

- Read the input file nameI.
- Generate all possible acronyms and select the ones composed of only small letters and digits or capital letters and digits. In both cases letters and digits must appear alternately along the string (e.g., “a2b6d” or “9A7X2Y0”).
- Store all selected acronyms in the output file nameO.

For example, given the following input file:

```
3
A1
Xy2
123B
```

the function should generate and write in the output file the following acronyms: A2B, 1X1, 1X2, 1X3, 1y1, 1y2, 1y3.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.